N047 Cloud Run에서 AI서비스 운영하기_해설서

Chapter01

/강의정리 챕터 01. Cloud Run에서 AI 서비스 운영하기

1. 학습 목적

AI 서비스를 만드는 것과 실제 사용자에게 공개해 운영하는 것은 다른 문제입니다.

서비스를 공개하면 답변이 중간에 끊기거나, 사용자가 갑자기 몰리거나, 예상보다 비용이 많이 발생할 수 있습니다. 새 버전을 배포한 뒤 기존 대화나 캐릭터 설정이 사라지는 문제도 생길 수 있습니다.

이번 강의의 목적은 Cloud Run에서 AI 서비스를 안정적으로 운영하고, 문제가 생겼을 때 로그를 통해 원인을 찾아내는 방법을 배우는 것입니다.

2. 중학생도 이해할 수 있는 설명

AI 서비스를 작은 식당이라고 생각해 보겠습니다.



① 요청 제한 시간: 손님을 기다리게 할 수 있는 시간

AI는 답변을 만드는 데 시간이 오래 걸릴 수 있습니다. 식당에서 주문한 음식이 너무 늦게 나오면 손님이 떠나듯, 요청 제한 시간이 너무 짧으면 답변이 끝나기 전에 요청이 종료될 수 있습니다.



② 인스턴스 수: 동시에 일할 수 있는 직원 수

인스턴스는 AI 서비스를 실행하는 서버 실행 단위입니다. 손님이 많으면 더 많은 인스턴스가 필요할 수 있지만, 무작정 늘리면 서버 운영비와 AI API 비용이 커질 수 있습니다.



③ 로그: 식당에서 발생한 일을 기록한 장부

어떤 요청이 들어왔는지, 오류가 발생했는지, 처리에 얼마나 걸렸는지 기록을 확인하면 문제의 원인을 찾는 데 도움이 됩니다.

핵심은 서버를 실행하는 것만으로 끝나는 것이 아니라, 사용자 수와 AI의 처리 특성에 맞게 운영 기준을 정하는 것입니다.

3. 핵심 개념 정리

개념	쉬운 설명	왜 필요한가요?
Cloud Run	웹 서비스나 AI 서비스를 실행하는 클라우드 환경	서버를 직접 관리하는 부담을 줄여 줍니다.
요청 타임아웃	요청 하나를 처리할 수 있는 최대 시간	긴 AI 답변이 중간에 끊기는 문제에 대응합니다.
인스턴스 수	서비스를 실행하는 서버 단위의 개수	요청이 몰릴 때 처리 능력과 비용을 조절합니다.
LLM API 비용	OpenAI, Google 등의 AI 모델을 이용하는 비용	요청이 늘면 사용량에 따라 비용이 증가할 수 있습니다.
로그	서비스에서 발생한 사건의 기록	오류 원인과 처리 상황을 파악합니다.
배포 버전	현재 서비스에 적용된 코드 버전	새 버전 배포 후 발생한 문제를 추적합니다.

여기서 주의할 점이 있습니다. Cloud Run의 요청 타임아웃과 LLM API의 처리 시간은 서로 다른 개념입니다. 타임아웃을 늘린다고 AI 모델이 반드시 빨라지는 것은 아닙니다.

또한 인스턴스 수를 제한한다고 LLM API 비용까지 자동으로 완전히 통제되는 것은 아닙니다. 사용량 제한과 비용 관리도 함께 고려해야 합니다.

4. 실제 서비스에 적용하는 예시

사용자가 공개된 AI 캐릭터 서비스에 접속해 질문한다고 가정하겠습니다.

사용자 질문 입력

예: "오늘 기분이 우울한데 위로해 줘."

Cloud Run

요청 접수 · 서버 실행 · 처리 시간 관리

LLM API 호출

AI 모델이 답변 생성 · API 사용 비용 발생

답변 반환 및 로그 확인

성공 여부 · 처리 시간 · 오류 기록

실제 운영에서는 다음과 같은 문제가 발생할 수 있습니다.

- 문제 A: AI 답변이 길어져 요청 제한 시간을 초과합니다.
- 문제 B: 단체 대화방에 주소가 퍼져 요청이 급증하고 비용이 증가합니다.
- 문제 C: 새 버전을 배포한 뒤 캐릭터의 설정이나 대화 기록이 사라집니다.

이때 무조건 코드를 수정하기보다는, 타임아웃 설정과 인스턴스 상태, 로그, 데이터 저장 구조 등을 차례로 확인해야 합니다.

특히 대화 기록을 서버 메모리에만 저장했다면 인스턴스가 바뀌거나 재시작될 때 기록이 사라질 수 있습니다. 중요한 대화 상태는 적절한 외부 데이터 저장소에 보관해야 합니다.

5. 내 프로젝트에 적용한다면?

현재 진행 중인 프로젝트 가운데서는 MSM(My-Story-Maker) 에 직접 적용하기 좋습니다.

MSM은 사용자의 글을 분석하고, 원래 문체를 보존하면서 이야기로 발전시키는 AI 서비스이므로 응답 시간과 데이터 보존이 중요합니다.

MSM 기능	예상되는 문제	운영 측면의 대응
글 분석하기	긴 글을 분석하다 시간 초과	입력 길이와 요청 제한 시간 점검

MSM 기능	예상되는 문제	운영 측면의 대응
AI 인터뷰	여러 질문을 처리하며 응답 지연	처리 시간과 오류 로그 확인
웹소설 변환	생성 시간이 길고 토큰 사용량 증가	생성 길이 제한과 비용 관리
스토리 보관함	배포 후 기존 자료가 보이지 않음	외부 DB 저장 및 배포 전후 확인
사용자 공개 서비스	요청 급증 또는 반복 호출	인스턴스 제한, 사용량 제한, 비용 알림 검토

이 강의에서 배운 운영 개념은 MSM뿐 아니라 AI 뉴스레터 에이전트나 중소기업 AI 진단 컨설팅 에이전트에도 활용할 수 있습니다.

6. AI 개발자·1인 창업가가 알아야 하는 이유

AI 서비스의 사업성은 답변 품질만으로 결정되지 않습니다.

- 사용자 경험: 답변이 너무 늦거나 자주 끊기면 사용자가 떠날 수 있습니다.
- 수익성: 사용량이 증가할수록 모델 호출 비용도 증가할 수 있습니다.
- 신뢰성: 배포 이후 사용자 데이터나 설정이 사라지면 신뢰를 잃습니다.
- 확장성: 사용자가 늘어도 서비스를 안정적으로 유지할 수 있어야 합니다.

따라서 1인 AI 사업가에게 필요한 역량은 코드를 작성하는 능력뿐 아니라, 서비스의 안정성과 비용을 관리하는 능력까지 포함합니다.

7. 오늘의 주관식 퀴즈

질문: 내가 만든 AI 서비스의 공개 주소를 단체 대화방에 올렸다고 상상해 보세요. 그날 밤 서비스에 어떤 일이 생길 수 있을지, 걱정되는 일을 세 가지 적어 보세요.

다음은 답변을 구성하는 데 참고할 수 있는 예시입니다.

1. 서비스 안정성: 사용자가 한꺼번에 몰려 답변이 느려지거나 요청이 시간 초과될 수 있습니다.
2. 비용 증가: 반복적인 요청이나 긴 답변 때문에 LLM API 비용이 예상보다 많이 발생할 수 있습니다.
3. 데이터 및 배포 문제: 서버가 재시작되거나 새 버전이 배포되면서 저장하지 않은 대화 내용이나 캐릭터 설정이 사라질 수 있습니다.

이 예시는 참고용입니다. 먼저 본인의 생각으로 세 가지를 작성해 보시면 좋겠습니다.

나의 답변 작성하기

걱정 1

걱정 2

걱정 3

답변 검토하기

8. 반드시 기억할 것

- AI 서비스는 일반 웹앱보다 응답 시간이 불규칙하고 외부 모델 사용 비용이 발생할 수 있습니다.
- 요청 타임아웃, 인스턴스 수, 사용량 제한을 서비스 특성에 맞게 설정해야 합니다.
- 문제가 생겼을 때는 로그를 통해 실제 발생한 일을 확인해야 합니다.
- 배포와 인스턴스 변경에 대비해 중요한 대화 데이터는 지속적으로 보관할 수 있는 구조로 설계해야 합니다.

9. 한 문장 요약

Cloud Run에서 AI 서비스를 운영한다는 것은 단순히 서비스를 공개하는 것이 아니라, 응답 시간·서버 자원·AI 사용 비용·데이터 보존을 관리하면서 사용자가 안정적으로 이용할 수 있도록 만드는 것입니다.

Chapter02

/강의정리 챕터 02. 시작 상태 맞추기

1. 학습 목적

이번 강의의 목적은 Cloud Run에서 AI 서비스를 운영하기 전에, 실습에 필요한 클라우드 환경이 올바르게 준비되어 있는지 확인하는 것입니다.

챕터 01에서는 AI 서비스를 운영할 때 응답 시간, 서버 자원, 비용, 로그를 관리해야 한다는 점을 배웠습니다. 이번 챕터에서는 그 작업을 시작하기 위한 준비 상태를 점검합니다.

핵심은 새로운 서비스를 만드는 것이 아니라, 이미 만들어진 서비스의 상태를 확인하는 것입니다.

2. 중학생도 이해할 수 있는 설명

AI 서비스를 온라인 식당이라고 생각해 보겠습니다.



① 프로젝트와 리전 확인

어느 식당의 어느 지점에서 일할 것인지 확인하는 과정입니다. 프로젝트와 리전이 잘못 설정되면 엉뚱한 환경을 조회하거나 서비스를 찾지 못할 수 있습니다.



② Cloud Run 서비스 확인

식당이 실제로 문을 열었는지 확인하는 것입니다. 이번 실습에서는 `friend-agent`라는 AI 서비스가 배포되어 있는지, 접속 주소가 있는지 확인합니다.



③ Secret Manager 확인

식당 직원만 알아야 하는 출입 비밀번호를 안전한 금고에 보관하는 것과 같습니다. AI 서비스가 LLM API를 이용할 수 있도록 API 키를 안전하게 보관합니다.

세 가지가 모두 준비되어 있어야 다음 실습을 원활하게 진행할 수 있습니다.

3. 핵심 개념 정리

개념	쉬운 설명	이번 실습에서의 역할
Google Cloud 프로젝트	클라우드 자원을 모아 관리하는 작업 공간	어느 환경을 사용할지 결정
gcloud CLI	명령어로 Google Cloud를 관리하는 도구	현재 설정과 서비스 상태 확인
리전(Region)	클라우드 자원이 위치한 지역	이번 실습은 `us-central1` 사용
Cloud Run	컨테이너 기반 서비스를 실행하는 환경	`friend-agent` 실행
서비스 URL	배포된 서비스에 접속하는 주소	브라우저에서 AI 서비스 확인
Secret Manager	API 키 같은 민감한 정보를 보관하는 서비스	`llm-api-key` 보관 및 접근 관리
서비스 계정	서비스가 다른 클라우드 자원에 접근할 때 사용하는 신원	AI 서비스가 시크릿을 읽도록 권한 부여

여기서 특히 구분해야 할 것이 있습니다.

- API 키를 보관하는 것과 서비스가 API 키를 읽을 권한을 갖는 것은 서로 다른 작업입니다.
- 시크릿이 존재하더라도 서비스 계정에 적절한 권한이 없으면 서비스를 실행할 때 키를 읽지 못할 수 있습니다.
- 시크릿이 존재한다는 사실만으로 API 키가 유효하거나 AI 호출이 정상 작동한다고 보장할 수는 없습니다.

4. 실습 명령어 이해하기

이번에는 다음 세 가지 명령어를 사용합니다.

① 현재 gcloud 설정 확인

```
gcloud config list
```

현재 선택된 계정, 프로젝트, 기본 리전 등의 설정을 확인합니다. 프로젝트 ID와 리전이 실습 환경에 맞는지 살펴봅니다.

② Cloud Run 서비스 URL 확인

```
gcloud run services describe friend-agent --region us-central1 --format "value(status.
url)"
```

지정한 리전에서 friend-agent 서비스의 URL을 조회합니다. 서비스가 없거나 프로젝트·권한·리전이 잘못되었다면 오류가 발생할 수 있습니다.

③ 시크릿 존재 여부 확인

```
gcloud secrets describe llm-api-key --format "value(name)"
```

시크릿의 메타데이터에서 이름을 확인합니다. API 키의 실제 값은 출력하지 않습니다.

이 세 명령어는 조회용 명령어입니다. 서비스나 시크릿을 새로 만들거나 설정을 변경하지 않습니다.

다만 `gcloud config list` 결과에는 계정이나 프로젝트 관련 정보가 포함될 수 있으므로, 결과를 다른 사람에게 공유할 때는 민감한 정보가 없는지 확인하는 것이 좋습니다.

5. 처음부터 준비해야 하는 경우

이미 실습 환경이 준비되어 있다면 이 절차를 반복할 필요가 없습니다. 준비가 안 된 경우에는 다음 순서로 진행합니다.

1. Google Cloud 기본 환경 준비

   무료 체험 가입, 프로젝트 생성, gcloud 설치와 초기화를 진행합니다.

2. 기본 설정 구성

   기본 리전을 `us-central1`로 설정하고, 시작 상태 문서에 안내된 API를 활성화합니다.

3. 실습 저장소 준비

   템플릿 저장소를 바탕으로 본인 실습용 저장소를 만들고 코드를 내려받습니다.

4. 시크릿과 권한 설정

   Secret Manager에 `llm-api-key`를 만들고, 해당 시크릿을 읽을 수 있도록 서비스 계정 권한을 설정합니다.

5. Cloud Run 배포 및 테스트

   `friend-agent`를 배포하고 서비스 URL을 열어 캐릭터를 만든 뒤 메시지를 보내 정상 응답을 확인합니다.

자세한 준비 절차는 강의에서 제공한 실습별 시작 상태 문서의 A, B 항목을 기준으로 진행하면 됩니다.

6. AI 개발자·1인 창업가가 알아야 하는 이유

이번 강의에서 중요한 습관은 작업하기 전에 현재 상태부터 확인하는 것입니다.

예를 들어 MSM(My-Story-Maker)을 배포한다고 생각해 보겠습니다.

- 잘못된 프로젝트를 선택하면 의도하지 않은 환경을 수정할 수 있습니다.
- API 키가 코드에 직접 들어가면 저장소가 공개될 때 키가 노출될 위험이 있습니다.
- 시크릿은 있지만 권한이 없다면 서비스가 API 키를 가져오지 못할 수 있습니다.
- 배포 URL이 있더라도 실제 AI 응답이 정상인지 별도로 테스트해야 합니다.

특히 비개발자 창업가가 코딩 에이전트를 활용할 때는 작업 범위를 명확하게 정하는 것이 중요합니다.

이번 강의의 요청처럼 다음과 같이 지시할 수 있습니다.

> 현재 설정과 서비스 상태만 확인해 주세요. 시크릿 값은 읽거나 출력하지 말고, 아무것도 만들거나 변경하지 마세요.

이것은 단순한 명령어 사용법을 넘어, AI 에이전트가 클라우드 환경을 다룰 때 불필요한 변경을 방지하는 안전한 작업 방식이기도 합니다.

7. 반드시 기억할 것

- `gcloud config list`는 현재 CLI 설정을 확인합니다.
- `gcloud run services describe`는 지정한 Cloud Run 서비스의 정보를 조회합니다.
- `gcloud secrets describe`는 시크릿의 메타데이터를 확인하며, 실제 키 값 확인과는 다릅니다.
- 시크릿 보관, 접근 권한 설정, 실제 API 호출 성공은 각각 별도로 확인해야 합니다.
- 이미 준비된 환경이라면 새로 만들지 말고 먼저 상태를 확인합니다.

8. 한 문장 요약

Cloud Run에서 AI 서비스를 운영하기 위한 첫 단계는 프로젝트·리전·배포 서비스·시크릿의 현재 상태를 안전하게 확인하고, 준비가 완료된 환경에서 다음 실습으로 넘어가는 것입니다.

Chapter03

/강의정리 챕터 03. 스트리밍과 요청 타임아웃

1. 학습 목적

이번 강의의 핵심은 AI가 답변을 생성하는 시간과 요청 전체를 처리하는 시간을 구분하고, 실제 측정값을 바탕으로 요청 타임아웃을 설정하는 것입니다.

AI 서비스는 답변을 생성하는 동안 사용자에게 결과를 조금씩 보여 줄 수 있습니다. 하지만 화면에 답변이 모두 표시되었다고 해서 서버의 작업까지 끝난 것은 아닙니다.

이번 실습에서는 다음 세 가지를 배웁니다.

- 스트리밍을 통해 사용자가 답변을 기다리는 시간을 줄이는 방법
- 요청 타임아웃 때문에 답변이 잘리거나 후속 작업이 중단되는 상황
- Cloud Run의 측정항목과 로그를 활용해 적절한 타임아웃을 결정하는 방법

2. 중학생도 이해할 수 있는 설명

2-1. 스트리밍은 피자를 한 조각씩 내놓는 것과 같습니다



식당에서 피자 한 판을 전부 만든 뒤 내놓을 수도 있고, 준비된 조각부터 손님에게 내놓을 수도 있습니다.

AI의 스트리밍(Streaming) 은 두 번째 방식입니다. AI가 답변을 생성하는 대로 조금씩 사용자 화면에 전달합니다.

전체 생성 시간이 같더라도 사용자는 첫 글자를 일찍 볼 수 있어 기다리는 느낌이 줄어듭니다.

예를 들어 AI가 다음 문장을 만든다고 가정해 보겠습니다.

> 내일 서울은 맑다가 오후부터 비가 내릴 예정입니다.

스트리밍을 사용하면 문장 전체가 완성되기 전부터 다음처럼 표시될 수 있습니다.

`내일 서울은 맑다가…` → `오후부터 비가…` → `내릴 예정입니다.`

그런데 중간에 연결이 끊기면 첫 번째 부분만 화면에 남을 수 있습니다. 이 경우 사용자는 실제 의미와 다른 정보를 받아들일 위험이 있습니다.

2-2. 타임아웃은 작업 마감 시간입니다

Cloud Run에 요청 타임아웃을 3초로 설정했다고 가정해 보겠습니다.

예시: 요청 전체 처리 시간 4초

0~0.5초

첫 글자 표시

0.5~2.5초

답변 스트리밍

2.5~3초

기억 정리 작업 진행 중

3초: 요청 타임아웃

답변이 완성되었더라도 서버의 후속 작업이 끝나기 전에 요청이 종료될 수 있습니다.

위 시간 분배는 원리를 설명하기 위한 가상 예시이며, 실제 측정값은 서비스마다 다릅니다.

여기서 중요한 것은 답변을 화면에 보내는 시간과 요청 전체가 끝나는 시간은 다를 수 있다는 점입니다.

이번 실습의 `friend-agent`는 답변을 스트리밍으로 전달한 뒤, 같은 요청 안에서 대화 내용을 요약해 기억으로 저장합니다. 따라서 화면에 답변이 모두 보이더라도 기억 저장 작업이 완료되지 않았을 수 있습니다.

3. 핵심 개념 정리

개념	쉬운 설명	이번 실습에서의 의미
스트리밍	답변을 조금씩 전달하는 방식	첫 글자를 빨리 보여 줌
첫 글자 시간	첫 번째 응답이 화면에 도착할 때까지의 시간	사용자가 기다리기 시작하는 구간
답변 전체 시간	답변 생성과 전달이 끝날 때까지의 시간	화면에 답변이 완성되는 시점
요청 전체 시간	요청 처리부터 후속 작업까지 걸리는 시간	타임아웃 설정에 필요한 핵심 지표
요청 타임아웃	요청 하나를 처리할 수 있는 최대 시간	너무 짧으면 요청이 중단될 수 있음
백분위수	여러 요청의 처리 시간을 순서대로 비교한 값	느린 요청이 얼마나 오래 걸리는지 파악

반드시 구분해야 할 세 가지 시간

1. 첫 글자 시간: 사용자가 언제부터 답변을 볼 수 있는가?
2. 답변 전체 시간: AI가 사용자에게 보낼 답변을 언제 완성하는가?
3. 요청 전체 시간: 답변 전달 이후의 작업까지 언제 끝나는가?

타임아웃을 결정할 때는 답변 전체 시간만 확인해서는 안 됩니다. 대화 기억 저장처럼 답변 뒤에 수행하는 작업이 있다면 그 시간까지 고려해야 합니다.

4. 실습: 요청 타임아웃을 3초로 줄였다가 복원하기

이번 실습은 일부러 장애 상황을 만들어 보는 과정입니다. 실제 서비스에 적용할 때는 사용자가 없는 실습 환경에서 진행하는 것이 안전합니다.

4-1. 먼저 현재 처리 시간을 확인합니다

Cloud Run에 배포된 `friend-agent`에 접속해 메시지를 몇 번 보내고 화면 오른쪽의 이번 턴 패널을 확인합니다.

- 첫 글자가 표시될 때까지 몇 초 걸리는지
- 답변 전체가 표시될 때까지 몇 초 걸리는지
- 요청 전체가 끝날 때까지 몇 초 걸리는지

강의의 로컬 환경 예시는 요청 전체가 약 3.5~4.5초입니다. 다만 이 수치는 예시이므로 본인의 Cloud Run 서비스에서 실제로 측정해야 합니다.

4-2. 요청 타임아웃을 3초로 변경합니다

```
gcloud run services update friend-agent \
  --region us-central1 \
  --timeout 3
```

이 명령은 `friend-agent`의 요청 타임아웃을 3초로 변경합니다. 다른 설정을 의도적으로 바꾸는 명령은 아니지만, 서비스 업데이트가 이루어지므로 실습 환경을 확인한 뒤 실행합니다.

변경 후에는 메시지를 여러 번 보내 다음 현상이 나타나는지 확인합니다.

- 답변이 문장 중간에서 끊김
- 답변은 완성되어 보이지만 `응답이 끝까지 오지 않았어요`라는 안내가 표시됨
- 요청 전체 시간이 3초 부근에서 제한됨

요청마다 처리 시간이 다르기 때문에 모든 요청에서 같은 현상이 발생하지는 않을 수 있습니다.

4-3. 실습이 끝나면 타임아웃을 복원합니다

```
gcloud run services update friend-agent \
  --region us-central1 \
  --timeout 300
```

실습의 기본값인 300초로 되돌립니다.

주의할 점은 300초가 모든 AI 서비스에 적합한 최적값이라는 뜻은 아니라는 것입니다. 실제 서비스에서는 측정한 처리 시간, 허용 가능한 대기 시간, 외부 API의 지연, 후속 작업의 소요 시간 등을 함께 고려해야 합니다.

5. 측정항목에서 내 서비스의 처리 시간 읽기

개별 메시지의 처리 시간은 이번 턴 패널에서 확인할 수 있지만, 여러 사용자의 요청을 종합적으로 판단하려면 Cloud Run의 측정항목(Metrics) 을 확인해야 합니다.

① Cloud Run 서비스 열기

Google Cloud Run 콘솔에서 실습 프로젝트를 선택하고 `friend-agent` 서비스를 엽니다.

② 측정항목 탭 선택

측정항목(Metrics) 탭으로 이동하고, 기간을 최근 1시간 또는 6시간으로 설정해 방금 보낸 요청이 포함되도록 합니다.

③ 요청 지연 시간 그래프 확인

Request latencies 그래프에서 50번째 백분위수와 99번째 백분위수를 차례로 확인합니다. 그래프에 표시되는 선 위에 마우스를 올리면 시각별 값을 볼 수 있습니다.

백분위수를 쉽게 이해하기

요청 100건을 처리 시간 순서대로 정렬했다고 생각해 보겠습니다.

P50 — 50번째 백분위수

중간 수준

요청의 약 절반은 이 값 이하의 시간에 처리됩니다. 일반적인 응답 속도를 파악하는 데 유용합니다.

P95 — 95번째 백분위수

느린 요청까지 고려

요청의 약 95%가 이 값 이하의 시간에 처리됩니다. 나머지 약 5%는 이보다 오래 걸릴 수 있습니다.

P99 — 99번째 백분위수

매우 느린 요청

요청의 약 99%가 이 값 이하의 시간에 처리됩니다. 느린 요청의 영향을 파악하는 데 유용합니다.

강의에서는 P50과 P99를 중심으로 비교합니다. P50만 보면 대부분의 요청은 빨라 보여도 일부 요청이 매우 오래 걸리는 문제를 놓칠 수 있기 때문입니다.

단, P99는 가장 느린 요청의 시간과 같은 뜻이 아닙니다. 실제 최댓값은 P99보다 클 수 있습니다. 또한 측정된 요청이 적거나 최근 요청이 그래프에 충분히 반영되지 않았다면 백분위수를 신중하게 해석해야 합니다.

6. AI 개발자·1인 창업가가 알아야 하는 이유

이 개념은 MSM(My-Story-Maker)처럼 긴 글을 분석하고 이야기로 변환하는 서비스에서 특히 중요합니다.

실제 기능	타임아웃이 짧을 때 생길 수 있는 문제	운영 시 고려할 점
글 분석	분석 결과가 중간에 끊김	입력 길이와 처리 시간 측정
AI 인터뷰	질문과 답변이 정상적으로 완료되지 않음	대화 단계별 시간 확인
웹소설 생성	긴 결과물을 생성하다 요청 종료	생성 길이 및 처리 구조 검토
대화 기억 저장	화면의 답변은 보이지만 기억이 저장되지 않음	후속 작업 완료 여부 확인

특히 기억 저장처럼 사용자에게 직접 보이지 않는 작업이 중요합니다. 화면의 답변만 보고 성공이라고 판단하면 실제 데이터가 저장되지 않은 문제를 발견하지 못할 수 있습니다.

서비스가 복잡해지면 답변 생성과 기억 저장을 분리하거나, 별도 작업 큐를 이용해 후속 작업을 처리하는 방법도 고려할 수 있습니다. 다만 이러한 구조 변경은 이번 실습의 필수 작업이 아니라 향후 개선 방향입니다.

7. 반드시 기억할 것

- 스트리밍은 답변을 조금씩 전달해 사용자가 첫 응답을 빨리 보도록 합니다.
- 답변이 화면에 완성되어도 요청 전체는 아직 끝나지 않았을 수 있습니다.
- 요청 타임아웃은 답변 생성뿐 아니라 같은 요청에서 실행되는 후속 작업에도 영향을 줄 수 있습니다.
- P50은 일반적인 처리 시간, P99는 느린 요청의 영향을 파악하는 데 유용합니다.

- 타임아웃은 짐작으로 정하기보다 실제 측정값을 근거로 설정해야 합니다.
- 실습을 위해 3초로 줄였다면 반드시 원래 설정인 300초로 복원합니다.

8. 한 문장 요약

AI 서비스의 요청 타임아웃은 답변 생성 시간만이 아니라 후속 작업을 포함한 실제 처리 시간을 측정하고, 느린 요청까지 고려해 충분한 여유를 두고 설정해야 합니다.

Chapter04

/강의정리 챕터 04. 인스턴스 수 제한

1. 학습 목적

이번 강의의 핵심은 Cloud Run 인스턴스 수를 제한해 비용을 관리하고, 인스턴스가 종료되거나 새로 생성될 때 AI 서비스의 데이터가 어떻게 되는지 이해하는 것입니다.

공개된 AI 서비스는 사용자가 요청할 때마다 LLM API 비용이 발생할 수 있습니다. 사용자가 갑자기 늘어나면 서버 자원과 API 사용량도 증가할 수 있으므로, 인스턴스 수에 적절한 상한을 설정해야 합니다.

이번 강의에서는 다음 세 가지를 배웁니다.

- 최소·최대 인스턴스 수를 설정하는 방법
- 콜드 스타트가 발생하는 이유와 비용의 관계
- 서버 메모리에 데이터를 저장할 때 생기는 문제와 데이터베이스가 필요한 이유

2. 중학생도 이해할 수 있는 설명

2-1. 인스턴스는 식당에서 일하는 직원과 같습니다







4

AI 서비스를 식당이라고 생각해 보겠습니다.

- 최소 인스턴스는 손님이 없어도 대기하는 직원 수입니다.
- 최대 인스턴스는 손님이 아무리 많아도 투입할 수 있는 직원 수의 상한입니다.
- 콜드 스타트는 직원이 모두 퇴근한 뒤 손님이 왔을 때, 다시 직원을 불러 일을 시작하는 과정입니다.

이번 실습의 설정은 다음과 같습니다.

최소 인스턴스: 0

요청이 없으면 인스턴스가 0개까지 줄어들 수 있습니다.

최대 인스턴스: 2

설정한 상한에 따라 인스턴스 수를 제한합니다. 다만 인스턴스 수가 2개라고 해서 전체 요청량이나 API 비용이 자동으로 제한되는 것은 아닙니다.

최소 인스턴스를 0으로 설정하면 유휴 인스턴스의 비용을 줄일 수 있지만, 다시 요청이 들어왔을 때 첫 응답이 늦어질 수 있습니다. 반대로 최소 인스턴스를 1로 유지하면 시작 지연을 줄이는 데 도움이 되지만 추가 비용이 발생할 수 있습니다.

3. 핵심 개념 정리

개념	의미	주의할 점
최소 인스턴스	유지하도록 설정한 인스턴스 수	0이면 유휴 상태에서 인스턴스가 없어질 수 있음
최대 인스턴스	인스턴스 수의 상한	순간적인 요청 증가를 처리하지 못할 수 있음
콜드 스타트	인스턴스가 없거나 준비되지 않은 상태에서 새 실행 환경을 준비하는 과정	첫 응답이 늦어질 수 있음
리비전(Revision)	Cloud Run 서비스의 특정 배포 버전	설정 변경 시 새 리비전이 생성될 수 있음
서버 메모리	실행 중인 인스턴스 내부의 임시 저장 공간	인스턴스가 교체되면 데이터가 사라질 수 있음
외부 데이터베이스	서비스 실행 환경과 분리된 데이터 저장소	여러 인스턴스에서 공통 데이터를 관리할 수 있음

여기서 가장 중요한 것은 인스턴스 수와 데이터 저장 위치는 별개의 설계 문제라는 점입니다.

인스턴스를 1개만 사용하더라도 그 인스턴스가 교체되면 메모리 속 데이터가 사라질 수 있습니다. 반대로 인스턴스가 2개 이상이어도 외부 데이터베이스를 사용하면 여러 인스턴스가 같은 데이터를 조회하고 갱신할 수 있습니다.

4. 실습: 최소 0, 최대 2로 설정하기

4-1. 직접 명령어로 실행하기

```
gcloud run services update friend-agent \
  --region us-central1 \
  --min-instances 0 \
  --max-instances 2
```

이 명령은 `friend-agent` 서비스의 최소 인스턴스를 0, 최대 인스턴스를 2로 설정합니다.

코딩 에이전트에게 맡길 때는 다음처럼 지시하면 됩니다.

> friend-agent 서비스의 최소 인스턴스는 0, 최대 인스턴스는 2로 설정해서 다시 배포해줘. 다른 설정은 바꾸지 마.

설정이 반영되었는지는 Cloud Run 콘솔의 서비스 상세 화면에서 리비전 탭을 열고, 가장 최근 리비전의 설정을 확인합니다.

4-2. 콜드 스타트 관찰하기

1. Cloud Run에서 `friend-agent`의 측정항목(Metrics) 탭을 엽니다.
2. 컨테이너 인스턴스 수(Container instance count) 그래프를 찾습니다.
3. 기간을 최근 1시간으로 설정합니다.
4. 약 10분 동안 서비스를 사용하지 않은 뒤 다시 메시지를 보냅니다.
5. 인스턴스 수가 0에 가까운 상태에서 다시 1 이상으로 증가하는지, 첫 답변이 늦게 시작되는지 확인합니다.

실제 그래프에서는 유휴·실행 중 인스턴스가 구분되어 보일 수 있습니다. 또한 트래픽, 다른 설정, 관측 시점에 따라 인스턴스 수가 0으로 내려가는 모습이 반드시 나타나는 것은 아닙니다.

콜드 스타트는 단순히 화면이 늦게 뜨는 현상이 아니라, 요청을 처리할 실행 환경이 준비되는 데 시간이 필요한 상황입니다.

5. 왜 캐릭터와 대화가 사라졌을까요?

이번 강의에서 가장 중요한 부분입니다.

템플릿의 `friend-agent`는 캐릭터, 대화, 기억을 서버 인스턴스의 메모리에 저장합니다. 이 구조에서는 인스턴스가 바뀌거나 사라질 때 기존 데이터가 유지되지 않을 수 있습니다.

이전 리비전의 인스턴스

캐릭터 A · 대화 기록 · 기억

새 리비전 배포 또는 인스턴스 종료

새 인스턴스

새로운 메모리 · 기존 캐릭터와 대화 기록 없음

새 리비전이 배포되면 새 실행 환경에서 서비스가 시작될 수 있습니다. 이전 인스턴스의 메모리에만 저장했던 캐릭터와 대화 기록은 새 인스턴스로 자동 이전되지 않습니다.

또한 최대 인스턴스가 2라면 두 인스턴스가 서로 다른 메모리를 가질 수 있습니다. 같은 사용자의 요청이 서로 다른 인스턴스로 전달되면 한쪽에는 대화 기록이 있고 다른 쪽에는 없는 문제가 발생할 수 있습니다.

따라서 캐릭터, 대화, 기억처럼 계속 보존해야 하는 데이터는 인스턴스 메모리가 아니라 외부 데이터베이스에 저장하는 것이 바람직합니다.

이것은 MSM에도 직접 적용되는 중요한 설계 원칙입니다. 사용자가 작성한 원본 글과 AI가 변환한 이야기, 인터뷰 기록 등을 서버 메모리에만 보관하면 안 됩니다.

6. 객관식 퀴즈

질문: 에이전트 서비스의 요청 타임아웃만 바꿔 다시 배포했더니, 방금까지 대화하던 캐릭터가 사라지고 캐릭터 만들기 화면이 나왔습니다. 가장 알맞은 설명은 무엇인가요?

1. 요청 타임아웃을 바꾸면 Cloud Run이 저장된 데이터를 일부러 지운다.

2. 새 리비전은 새 인스턴스에서 실행되고, 캐릭터와 대화는 이전 인스턴스의 메모리에만 있었다.

3. 브라우저가 캐시를 지워서 화면이 처음 상태로 돌아갔다.

4. 최대 인스턴스가 2라서 데이터를 2개까지만 보관한다.

정답 확인

7. 콜드 스타트를 줄이는 방법과 대가

선택	장점	단점
최소 인스턴스 0	유휴 인스턴스 비용을 줄일 수 있음	첫 요청이 늦게 시작될 수 있음
최소 인스턴스 1	시작 지연을 줄이는 데 도움	인스턴스 유지 비용 발생
컨테이너 이미지 최적화	시작에 필요한 작업을 줄일 수 있음	이미지와 초기화 과정을 개선해야 함

수업용 서비스처럼 사용 빈도가 낮다면 최소 인스턴스 0으로 시작하고, 몇 초의 첫 응답 지연을 받아들이는 것이 합리적일 수 있습니다.

반면 고객이 언제든 접속하는 상용 서비스라면 첫 응답 속도와 비용을 함께 비교해야 합니다. 최소 인스턴스를 1로 설정해도 모든 지연이 사라지는 것은 아닙니다. 외부 LLM API의 응답 지연 등은 별도로 발생할 수 있습니다.

8. 내 프로젝트에 적용하기

현재 진행 중인 MSM을 예로 들면 다음과 같습니다.

- 원본 글(O): 사용자가 입력한 글이므로 외부 데이터베이스에 영구 저장합니다.
- 정리된 글(R): 사용자의 문체를 보존해 수정한 결과를 저장합니다.
- 웹소설(N): 긴 생성 작업의 결과를 저장하고, 실패하면 이어서 처리할 수 있는 구조를 검토합니다.
- 웹툰(W): 이미지와 관련 메타데이터를 인스턴스 메모리가 아닌 지속성 있는 저장소에 보관합니다.
- 대화 및 기억: 여러 인스턴스에서 공통으로 조회할 수 있도록 외부 데이터베이스를 사용합니다.

이렇게 설계하면 인스턴스가 바뀌거나 서비스가 재배포되어도 사용자의 콘텐츠가 사라질 위험을 줄일 수 있습니다.

9. 반드시 기억할 것

- 최소 인스턴스는 대기 비용과 첫 응답 지연의 균형을 조절합니다.
- 최대 인스턴스는 서버 실행 단위의 상한이지, 전체 AI API 비용의 상한이 아닙니다.
- 콜드 스타트는 인스턴스가 준비되는 과정에서 발생하는 지연입니다.
- 인스턴스 메모리에만 저장된 데이터는 재배포나 인스턴스 교체 후 유지되지 않을 수 있습니다.
- 여러 인스턴스가 같은 데이터를 사용해야 한다면 외부 데이터베이스 같은 공유 저장소가 필요합니다.

10. 한 문장 요약

Cloud Run의 인스턴스 수 제한은 비용과 응답 속도를 조절하는 장치이며, 서비스가 재시작되거나 여러 인스턴스로 확장되어도 데이터를 보존하려면 중요한 상태를 외부 저장소에 관리해야 합니다.

Chapter05

/강의정리 챕터 05. 최대 인스턴스로 막지 못하는 비용

1. 학습 목적

이번 강의의 핵심은 서버의 처리량을 제한하는 것과 AI 사용 비용의 총액을 제한하는 것은 서로 다른 문제라는 점을 이해하는 것입니다.

Cloud Run에서 최대 인스턴스를 2개로 제한해도 각 인스턴스가 여러 요청을 동시에 처리할 수 있습니다. 따라서 LLM API 호출은 계속 누적되고, 예상보다 많은 비용이 발생할 수 있습니다.

이번 강의에서는 다음 다섯 가지를 배웁니다.

- 최대 인스턴스와 동시성의 관계
- LLM API 비용과 Cloud Run 비용의 차이

- 실제 측정값을 이용한 요청당 비용 계산
- LLM 제공사 사용 한도와 Google Cloud 예산 알림의 차이
- 사용자 자신의 API 키를 사용하는 BYOK 구조와 보안 원칙

2. 중학생도 이해할 수 있는 설명

2-1. 최대 인스턴스 2개는 직원 2명이 하루에 처리할 수 있는 주문 수를 제한하지 않습니다

식당에 직원이 두 명 있다고 생각해 보겠습니다.

- 최대 인스턴스 2개: 일할 수 있는 직원은 최대 두 명입니다.
- 동시성 80: 설정상 직원 한 명이 동시에 최대 80개의 요청을 처리할 수 있도록 허용합니다.
- LLM API 사용 한도: 식당에서 사용할 수 있는 한 달 식재료 예산과 비슷합니다.

단순한 계산으로 최대 인스턴스 2개와 인스턴스당 동시성 80을 적용하면 동시에 처리할 수 있는 요청 수는 최대 160개 수준입니다.

하지만 이것은 한 번에 처리할 수 있는 요청 수를 설명하는 것이지, 하루 또는 한 달 동안 처리할 수 있는 총요청 수를 제한하는 것은 아닙니다.

2개

최대 인스턴스

80

인스턴스당 동시성 설정

최대 160개 수준의 동시 요청

실제 처리량은 요청 시간, 자원, 설정 등에 따라 달라집니다.

예를 들어 요청 하나의 처리 시간이 8초라면, 단순화한 계산에서 시간당 처리 가능한 요청은 다음과 같습니다.

$$
160 \times \frac{3,600}{8} = 72,000
$$

즉, 최대 인스턴스가 2개여도 요청이 계속 들어오면 한 시간에 7만 건이 넘는 요청을 처리할 수 있는 규모가 됩니다. 실제 처리량은 동시성 설정과 요청 분포, 인스턴스의 처리 능력 등에 따라 달라집니다.

결론: 최대 인스턴스는 동시에 일할 수 있는 규모를 제한하지만, 시간이 지나면서 누적되는 LLM API 호출 총량을 직접 제한하지는 않습니다.

3. 비용을 통제하는 세 가지 장치

장치	막거나 관리하는 것	막지 못하는 것
Cloud Run 최대 인스턴스	인스턴스 수와 이에 따른 처리 규모	LLM API의 누적 비용 총액
LLM 제공사의 사용 한도	해당 계정·키에 적용되는 LLM API 사용 한도	Cloud Run 등 다른 서비스의 비용
Google Cloud 예산 알림	예산 사용 상황을 알림으로 전달	지출 자체를 자동으로 중단하는 것

이 세 가지를 혼동하지 않는 것이 중요합니다.

3-1. LLM 제공사의 월 사용 한도

OpenAI나 Google 등 LLM 제공사의 계정에서 사용량 한도나 지출 한도를 설정하면, 해당 제공사의 정책에 따라 한도 도달 시 API 호출이 제한될 수 있습니다.

다만 모든 제공사가 동일한 방식으로 한도를 강제하는 것은 아닙니다. 실제로 어떤 한도 설정이 제공되는지, 한도가 언제 적용되는지, 초과 사용이 가능한지는 각 제공사의 계정과 요금제에 따라 확인해야 합니다.

또한 실습에서 회사가 제공한 API 키를 사용한다면, 수강생에게 한도 변경 권한이 없을 수 있습니다. 같은 키를 여러 사람이 공유하면 한도에 도달했을 때 다른 사용자의 서비스까지 영향을 받을 수 있습니다.

3-2. Google Cloud 예산 알림

Google Cloud의 예산 알림은 지출이 일정 수준에 도달했을 때 알려 주는 장치입니다.

예를 들어 예산의 50%에서 알림을 받도록 설정할 수 있습니다. 그러나 예산 알림은 자동으로 서비스 사용을 중단하는 비용 상한이 아닙니다.

따라서 상용 AI 서비스를 배포하기 전에는 LLM 제공사의 사용 한도와 Google Cloud의 예산 알림을 각각 확인해야 합니다.

4. 요청 100건당 비용 계산하기

이번 강의에서는 실제 측정값을 이용해 AI 서비스의 비용 구조를 이해합니다.

AI 서비스의 비용은 크게 두 부분으로 나뉩니다.

① Cloud Run 비용

서비스가 실행되는 동안 사용하는 CPU·메모리와 요청 수 등에 따라 발생합니다. 과금 방식과 무료 사용량은 서비스 설정과 적용되는 요금 정책에 따라 달라집니다.

② LLM API 비용

모델에 보내는 입력 토큰과 모델이 생성하는 출력 토큰 등에 따라 발생합니다. 모델별 단가와 입력·출력 토큰의 구분을 확인해야 합니다.

`friend-agent`는 메시지 하나를 처리하면서 LLM API를 세 번 호출합니다.

1. 도구 선택
2. 사용자에게 전달할 답변 생성
3. 대화 기억 정리

따라서 비용을 계산할 때는 답변 생성 한 번의 비용만 계산하면 안 됩니다. 세 번의 호출에서 사용한 토큰을 합산하거나, 각 호출별 단가와 토큰 수를 따로 계산해야 합니다.

실습 순서

1. Cloud Run의 `friend-agent` 서비스에서 측정항목(Metrics) 탭을 엽니다.
2. 실습한 시간대로 기간을 설정합니다.
3. 요청 수와 요청 지연 시간 그래프를 확인합니다.
4. 요청 지연 시간의 P50(50번째 백분위수)을 읽습니다.
5. 강의의 비용 계산 화면에 처리 시간과 LLM 제공사의 실제 토큰 단가를 입력합니다.
6. LLM API의 세 번 호출을 모두 포함해 요청 100건당 비용을 계산합니다.

강의 예시에서는 요청 100건당 Cloud Run 비용이 약 2센트이고, LLM API 비용이 그보다 10배 이상 클 수 있다고 설명합니다. 이는 예시 조건에 따른 값이며, 실제 비용은 모델과 토큰 수, 응답 길이, 처리 시간 및 요금 정책에 따라 달라집니다.

또한 Cloud Run 무료 사용량이 있더라도 모든 사용량이 항상 무료인 것은 아닙니다. 현재 적용되는 무료 사용량과 과금 조건을 확인해야 합니다.

5. 객관식 퀴즈 ① 최대 인스턴스

질문: 에이전트 서비스에 최대 인스턴스 2를 설정했습니다. 이 설정에 대한 설명으로 옳은 것은 무엇인가요?

1. 동시에 처리되는 요청이 최대 2건으로 제한된다.

2. 한 달 동안 LLM API 비용이 일정 금액을 넘지 않게 막아 준다.

3. 켜지는 인스턴스가 2개를 넘지 않지만, 인스턴스마다 여러 요청을 동시에 처리하므로 LLM API 호출 총량을 제한하지는 않는다.

4. 인스턴스가 항상 2개 켜져 있어서 콜드 스타트가 없어진다.

정답 확인

6. 사용자 자신의 API 키를 사용하는 BYOK 구조

지금까지는 운영자가 가진 API 키 하나로 모든 사용자의 요청을 처리했습니다. 이 구조에서는 사용자가 많아질수록 운영자가 부담하는 LLM API 비용도 증가합니다.

이 문제를 해결하는 한 가지 방법이 BYOK입니다.

BYOK(Bring Your Own Key) 는 사용자가 자신의 LLM API 키를 가져와 서비스에 연결하는 방식입니다.

운영자 키 방식

여러 사용자

AI 서비스

운영자 API 키

사용량 비용과 한도 관리 책임이 운영자에게 집중됩니다.

BYOK 방식

사용자 A의 키 · 사용자 B의 키

AI 서비스가 각 사용자의 키로 호출

각 사용자의 LLM API 사용량은 해당 사용자의 계정에 청구되며, 사용 한도도 사용자 계정에 따라 적용됩니다.

BYOK가 항상 더 좋은 선택인 것은 아닙니다. 서비스가 사용자의 브라우저가 닫힌 동안에도 AI 작업을 실행해야 하는지에 따라 키 보관 구조를 달리해야 합니다.

키 보관 방식 비교

항목	브라우저에서 전달	서버에 암호화해 저장
키 보관 위치	사용자의 브라우저	서버 측 데이터베이스
서버의 지속 작업	사용자가 접속하지 않으면 제약이 큼	사용자 접속이 없어도 가능

항목	브라우저에서 전달	서버에 암호화해 저장
구현 난이도	상대적으로 낮음	상대적으로 높음
주요 고려사항	전송 구간 보호, 로그 미노출, 브라우저 보안	암호화, 사용자별 접근 제어, 키 삭제, 암호화 키 관리

브라우저에서 전달하는 방식은 요청을 처리하는 동안 서버가 키를 사용하고 저장하지 않도록 설계할 수 있습니다. 반면 서버에 저장하는 방식은 사용자가 화면을 닫은 뒤에도 작업할 수 있지만, 키 보호와 삭제 정책을 더 엄격하게 설계해야 합니다.

MSM처럼 사용자가 화면을 닫은 뒤에도 예약된 작업이나 알림을 실행하려면 서버 측에서 안전하게 키를 관리하는 구조가 필요할 수 있습니다.

7. 사용자 API 키를 다룰 때 반드시 지킬 보안 원칙

코딩 에이전트에게 단순히 "사용자가 자기 API 키를 입력할 수 있게 해 줘"라고 요청하면 충분하지 않습니다. 키를 저장하거나 로그에 남기는 방식까지 명확히 지정해야 합니다.

- 요청 로그, 오류 메시지, 이번 턴 패널에 API 키가 기록되지 않도록 합니다.
- 키를 입력한 뒤 화면에 전체 키를 다시 표시하지 않습니다. 필요하면 끝 네 자리만 표시합니다.
- 잘못된 키나 사용 한도 초과로 API 호출이 거부되면 그 이유를 사용자에게 안내합니다.
- 사용자 키가 실패했다고 운영자 키로 몰래 대신 호출하지 않습니다.
- 사용자가 자신의 키를 삭제할 수 있는 기능을 제공합니다.
- 서버에 키를 저장한다면 암호화, 접근 권한 관리, 삭제 정책을 함께 설계합니다.

특히 다음 객관식 문제는 실제 개발에서 매우 중요합니다.

8. 객관식 퀴즈 ② 사용자 API 키와 로그

질문: 사용자가 화면에 자기 LLM API 키를 넣는 기능을 만들고 있습니다. 서버가 받은 요청을 디버깅하려고 요청 본문 전체를 로그에 남기도록 해 두었다면 어떤 문제가 생길까요?

1. 문제없다. 로그는 운영자만 볼 수 있다.

2. 요청 본문에 들어 있는 사용자 키가 로그에 그대로 남아, 로그를 볼 수 있는 사람이면 누구나 그 키로 사용자 계정의 LLM API를 호출할 수 있다.

3. Cloud Run이 키를 자동으로 가려서 남기므로 문제없다.

4. 로그가 커져서 요금이 늘어나는 것만 문제다.

정답 확인

9. 반드시 기억할 것

- 최대 인스턴스는 동시 처리 규모를 제한하지만 누적 API 비용의 상한은 아닙니다.
- LLM API 비용은 모델 호출 횟수와 입력·출력 토큰 수에 영향을 받습니다.
- Cloud Run 비용과 LLM 제공사 비용은 별도로 계산하고 관리해야 합니다.
- Google Cloud 예산 알림은 지출을 자동으로 차단하지 않습니다.
- BYOK는 비용 부담을 사용자 계정으로 분리할 수 있지만 키 보호 책임이 중요합니다.
- 사용자 API 키는 로그나 오류 메시지에 남기지 말고, 안전한 저장·사용·삭제 방식을 설계해야 합니다.

10. 한 문장 요약

Cloud Run의 최대 인스턴스는 서버 처리 규모를 제한하는 장치일 뿐이므로, AI 서비스의 비용을 통제하려면 LLM 제공사 사용 한도와 클라우드 예산을 별도로 관리하고 사용자 API 키도 안전하게 다뤄야 합니다.

Chapter06

/강의정리 챕터 06. 로그의 세 종류

1. 오늘 강의의 목적

오늘 강의의 핵심은 서비스에 문제가 생겼을 때, 추측하지 말고 로그를 근거로 원인을 찾아내는 방법을 배우는 것입니다.

AI 코딩 에이전트가 코드를 고치는 일을 도와줄 수는 있지만, 실제로 어떤 일이 일어났는지 확인하고 정확한 근거를 전달하는 일은 서비스를 운영하는 사람의 몫입니다.

예를 들어 AI 서비스가 갑자기 느려졌다고 가정해 보겠습니다.

- 막연한 요청: "AI가 느려요. 고쳐 주세요."
- 좋은 요청: "오늘 오후 2시부터 2시 10분 사이에 `friend-agent` 서비스에서 응답 시간이 5초를 넘은 요청이 있습니다. 해당 시간대의 요청 로그와 관련 컨테이너 로그를 확인하고, 원인으로 판단한 근거를 먼저 보여 주세요. 아직 코드는 수정하지 마세요."

두 번째 요청은 AI가 엉뚱한 부분을 수정할 가능성을 줄여 줍니다.

2. 중학생도 이해할 수 있는 핵심 개념

2-1. 로그란 무엇인가요?

로그는 서비스에서 일어난 일을 시간순으로 기록한 사건 일지입니다.

학교에서 사고가 발생했다고 생각해 보세요. 사고를 해결하려면 학생의 이야기뿐 아니라 출석 기록, CCTV, 교사의 관찰 기록 등을 함께 확인해야 합니다.

클라우드 서비스도 마찬가지입니다. 하나의 로그만 보면 전체 원인을 놓칠 수 있습니다.

2-2. 로그의 세 종류

① 요청 로그 (Request log)

Cloud Run이 자동으로 기록합니다.

사용자가 언제 요청했는지, 어떤 주소를 호출했는지, 응답 코드가 무엇인지, 얼마나 걸렸는지를 확인합니다.

예: `/api/chat` 요청, 상태 코드 `200`, 지연 시간 `2.4s`

② 컨테이너 로그 (Container log)

내가 만든 애플리케이션이 기록합니다.

앱 내부에서 어떤 작업을 했는지, 오류가 발생했는지 확인합니다.

예: `LLM_PROVIDER`에 지원하지 않는 값이 설정되어 앱이 시작되지 않음

③ 시스템 로그 (System log)

Cloud Run 플랫폼이 기록합니다.

인스턴스가 시작되거나 종료되는 과정, 컨테이너 시작 실패 등 플랫폼에서 일어난 일을 확인합니다.

예: 컨테이너가 정상적으로 시작되지 않아 인스턴스 실행에 실패함

이 세 종류의 로그는 로그 화면에 시간순으로 섞여 표시될 수 있습니다. 따라서 로그의 내용뿐 아니라 누가 기록한 로그인지도 구분해야 합니다.

참고로 빌드 과정에서 발생한 문제는 Cloud Run의 일반 로그가 아니라 Cloud Build 기록도 확인해야 합니다.

2-3. 문제가 생겼을 때 어떤 로그부터 볼까요?

증상	먼저 확인할 로그
답변이 너무 늦음	요청 로그의 지연 시간
서버가 오류를 반환함	요청 로그의 HTTP 상태 코드와 관련 컨테이너 로그
앱이 시작되지 않음	컨테이너 로그와 시스템 로그

증상	먼저 확인할 로그
배포 자체가 실패함	리비전 상태, 컨테이너 로그, 시스템 로그, 터미널 메시지
빌드가 실패함	Cloud Build 기록

기억할 원칙은 간단합니다. 사용자에게 보인 현상으로 시작하고, 그 현상을 설명하는 내부 로그를 찾아가야 합니다.

3. 비유로 이해하기: 병원 응급실의 사건 기록

AI 서비스를 병원 응급실이라고 생각하면 세 로그의 차이가 쉽게 이해됩니다.

요청 로그 = 접수 기록

환자가 언제 왔고, 얼마나 기다렸고, 어떤 결과를 받았는지 기록합니다.

컨테이너 로그 = 의료진의 진료 기록

어떤 검사와 처치를 했으며, 무엇이 잘못되었는지 기록합니다.

시스템 로그 = 병원 시설 관리 기록

전원이 꺼졌는지, 장비가 작동하지 않았는지 등 시설 차원의 문제를 기록합니다.

환자가 오래 기다렸다는 사실만으로는 원인을 알 수 없습니다. 접수 기록으로 대기 시간을 확인하고, 진료 기록과 시설 기록을 함께 살펴야 원인을 정확히 찾을 수 있습니다.

4. 실습 내용과 주요 명령어

4-1. 내 대화 한 번의 흔적 찾기

강의에서는 `friend-agent`에 대화를 한 번 보내고, 그 요청이 남긴 로그를 찾습니다.

1. 서비스에서 캐릭터에게 메시지를 보냅니다.
2. Cloud Run의 `friend-agent` 서비스에서 Logs 탭을 엽니다.
3. 해당 시각의 `POST /api/chat` 요청 로그를 찾습니다.
4. 상태 코드와 지연 시간을 확인합니다.
5. 앞뒤의 컨테이너 로그에서 `기억 정리` 같은 작업 기록을 확인합니다.
6. 로그에 표시된 리비전이 현재 최신 리비전과 같은지 확인합니다.

왜 리비전을 확인해야 할까요? 배포 직후에는 이전 버전과 새 버전의 로그가 함께 나타날 수 있기 때문입니다. 문제를 찾았더라도 다른 버전의 로그를 보고 있다면 원인을 잘못 판단할 수 있습니다.

4-2. 로그 탐색기에서 조건으로 검색하기

로그가 수백 줄 쌓이면 하나씩 읽기 어렵습니다. 이때 Google Cloud의 Logs Explorer에서 조건을 지정합니다.

찾을 내용	검색 조건
Cloud Run 서비스 로그	`resource.type="cloud_run_revision"`
`friend-agent` 서비스만	`resource.labels.service_name="friend-agent"`
오류 이상	`severity>=ERROR`
서버 오류 응답	`httpRequest.status>=500`
3초 초과 요청	`httpRequest.latency>"3s"`
특정 리비전	`resource.labels.revision_name="friend-agent-00007-xyz"`
특정 문구가 포함된 로그	`jsonPayload.message:"기억 정리"`

여러 조건을 함께 입력하면 일반적으로 모든 조건을 만족하는 로그를 찾습니다. `=`는 정확한 값 비교에, `:`는 해당 문자열이 포함된 항목을 찾는 데 사용합니다.

4-3. 터미널에서도 로그 확인하기

```
# 최근 로그 20줄 읽기
gcloud run services logs read friend-agent \
  --region us-central1 \
  --limit 20

# 지난 1시간 동안 발생한 오류 로그 읽기
gcloud logging read \
  'resource.type="cloud_run_revision" AND resource.labels.service_name="friend-agent"
AND severity>=ERROR' \
  --freshness 1h \
  --limit 20
```

여기서 `gcloud logging read`는 로그를 조회하는 명령입니다. 로그를 읽는 것과 서비스 설정을 변경하는 것은 다릅니다. 원인을 확인하기 전에는 조회부터 하고, 수정은 근거를 확보한 뒤에 진행하는 것이 안전합니다.

4-4. 일부러 실패하는 배포를 통해 원인 찾기

이번 실습에서는 `LLM_PROVIDER`를 지원하지 않는 값인 `unknown`으로 변경합니다.

```
gcloud run services update friend-agent \
  --region us-central1 \
  --update-env-vars LLM_PROVIDER=unknown
```

앱은 지원하지 않는 설정값 때문에 시작하지 못할 수 있습니다. 이때 확인할 순서는 다음과 같습니다.

1

리비전 탭: 새 리비전이 생성되었고 실패 상태인지 확인

2

컨테이너 로그: `LLM_PROVIDER` 관련 오류 메시지 확인

3

시스템 로그: 컨테이너 시작 실패 등 플랫폼의 기록 확인

4

요청 로그: 실제 요청이 어느 리비전에서 처리되었는지 확인

5

터미널: 배포 명령이 마지막에 출력한 결과 확인

실패한 리비전이 트래픽을 받지 않으면 이전 정상 리비전이 계속 요청을 처리할 수 있습니다. 다만 실제 동작은 트래픽 할당과 배포 상태에 따라 확인해야 합니다.

실습 후에는 반드시 원래 설정으로 복원합니다. 원래 `LLM_PROVIDER` 값이 `gemini`였다면 다음과 같이 되돌립니다.

```
gcloud run services update friend-agent \
  --region us-central1 \
  --update-env-vars LLM_PROVIDER=gemini
```

위의 `gemini`는 예시입니다. 실제로 사용하던 값이 다르다면 그 값으로 복원해야 합니다. 처음부터 해당 변수가 설정되어 있지 않아 앱의 기본값을 사용했다면 강의에서 안내한 `--remove-env-vars LLM_PROVIDER` 방식으로 되돌릴 수 있습니다.

주의: 의도적인 실패 배포는 실습용 서비스에서만 실행하세요. 운영 서비스에서는 사용자 요청에 영향을 줄 수 있습니다.

5. 코딩 에이전트에게 로그를 전달하는 방법

AI 코딩 에이전트가 문제를 해결하도록 하려면 다음 네 가지를 알려주는 것이 좋습니다.

- 증상: 사용자가 무엇을 경험했는가?

- 시간: 언제 발생했는가?
- 대상: 어떤 서비스와 리비전인가?
- 근거: 실제 로그에 무엇이라고 기록되어 있는가?

예를 들어 다음과 같이 요청할 수 있습니다.

`friend-agent` 서비스에서 오늘 14:00부터 14:10 사이에 배포가 실패했습니다. 실패한 리비전은 `friend-agent-00007-xyz`입니다.

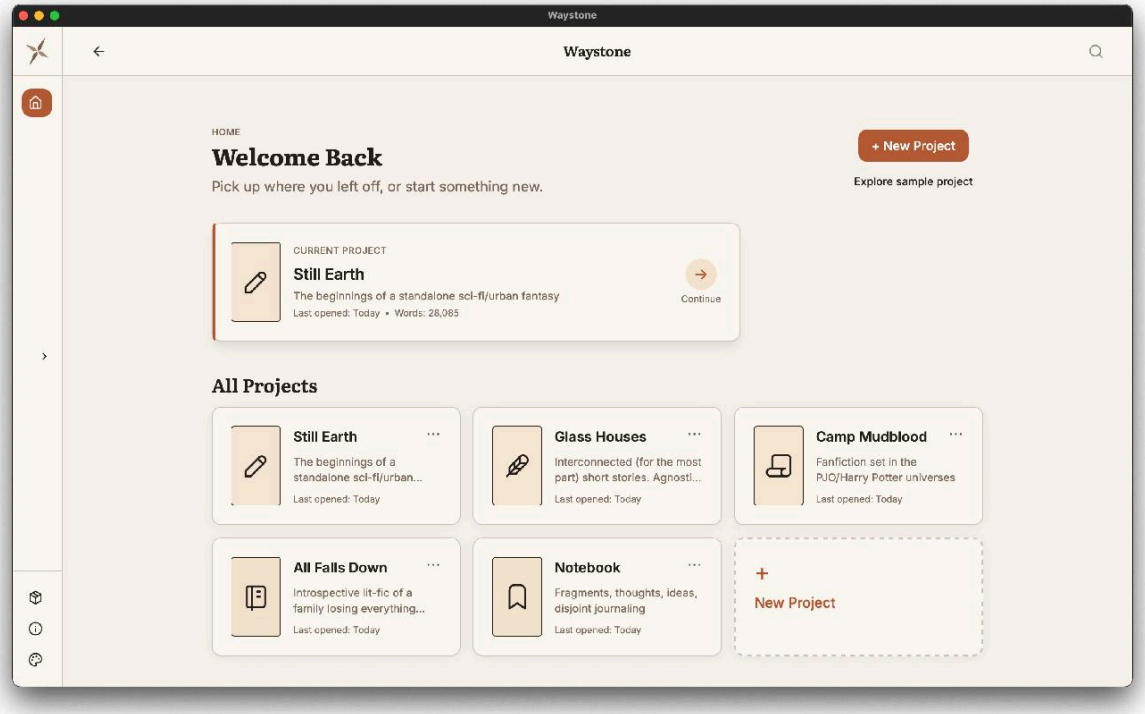
1. `gcloud logging read` 명령으로 해당 시간대의 이 서비스 로그만 조회해 주세요.
2. 원인이라고 판단한 로그 줄을 시각, 심각도, 리비전과 함께 그대로 보여 주세요.
3. 컨테이너 로그와 시스템 로그를 구분하고, 어떤 근거로 원인을 판단했는지 설명해 주세요.
4. 아직 코드나 설정을 수정하지 마세요.

제가 로그 탐색기에서 같은 기록을 확인한 다음 수정 여부를 결정하겠습니다.

이 방식은 에이전트가 추측으로 코드를 변경하는 일을 막는 데 도움이 됩니다. 또한 로그에 API 키, 인증 토큰, 사용자 이름이나 연락처가 포함되어 있지 않은지 확인한 뒤 전달해야 합니다.

6. 오늘 배운 내용을 내 프로젝트에 적용한다면

사용자님의 프로젝트 중에서는 MSM(My-Story-Maker) 과 뉴스레터 에이전트에 특히 유용합니다.



① MSM — My-Story-Maker

사용자가 글을 입력하고 AI가 문체를 분석하거나 이야기를 변환하는 과정에서 오류가 발생할 수 있습니다.

- 요청 로그: 글 분석 요청이 성공했는지, 얼마나 걸렸는지 확인
- 컨테이너 로그: 문체 분석이나 이야기 변환 단계에서 오류가 났는지 확인
- 시스템 로그: 서비스 인스턴스가 시작되지 않았는지 확인

특히 원문 글에는 개인적인 이야기가 담길 수 있으므로 로그에는 원문 전체 대신 작업 단계, 처리 시간, 문서 ID 같은 최소한의 정보만 남기는 것이 좋습니다.



② 뉴스레터 자동화 에이전트

뉴스 수집부터 선별, 조사, 검증, 발행까지 여러 단계가 연결되어 있습니다.

- 요청 로그: 실행 요청이 정상적으로 시작되고 끝났는지 확인
- 컨테이너 로그: 어느 단계에서 뉴스 수집이나 검증이 실패했는지 확인
- 시스템 로그: 작업 중 서버가 종료되거나 시작에 실패했는지 확인

`collect → select → research → verify → publish`처럼 단계별 기록을 남기면 문제가 발생한 위치를 좁히기 쉬워집니다.

두 프로젝트 모두 성공 여부만 기록하기보다 어느 단계에서, 어떤 이유로, 몇 건을 처리했는지 기록하면 운영과 개선이 훨씬 쉬워집니다. 단, 개인 데이터나 API 키 같은 민감한 내용은 기록하지 않는 것이 중요합니다.

7. 객관식 퀴즈 정답

정답: 2번

어제 저녁에 답변이 늦었다면 해당 시간대의 요청 로그를 찾아 지연 시간 조건으로 걸러야 합니다.

이유는 다음과 같습니다.

- 요청 로그에서 실제로 얼마나 오래 걸렸는지 확인할 수 있습니다.
- 컨테이너 로그는 내부 작업의 원인을 추적하는 데 사용합니다.
- 오류가 발생하지 않고 정상 응답 코드 `200`을 반환했더라도 응답이 느릴 수 있으므로, `ERROR` 로그만 검색하면 문제를 놓칠 수 있습니다.
- Cloud Build 기록은 빌드 문제를 확인하는 곳이지, 사용자의 대화가 늦어진 이유를 가장 먼저 확인하는 곳은 아닙니다.

8. 주관식 퀴즈 모범 답안

질문: 터미널 메시지만 보고 원인을 짐작했다면 어떤 오해를 할 수 있으며, 실제 원인은 어느 로그 줄에서 확인할 수 있을까요?

모범 답안:

터미널의 마지막 메시지는 배포가 실패했다는 결과만 알려줄 수 있기 때문에, 이를 보고 배포 명령 자체나 Cloud Run 플랫폼에 문제가 있다고 오해할 수 있습니다. 그러나 실제 원인은 앱이 시작되는 과정에서 발생한 설정 오류일 수 있습니다.

이번 실습에서는 `LLM_PROVIDER`에 지원하지 않는 `unknown` 값이 설정된 것이 원인입니다. 이 사실은 컨테이너 로그의 `ERROR` 메시지에서 확인하고, 해당 로그의 시각과 리비전 정보를 시스템 로그 및 리비전 상태와 비교해 실패 과정이 일치하는지 검증해야 합니다. 터미널 메시지는 결과를, 컨테이너 로그는 직접적인 원인을, 시스템 로그는 플랫폼에서 나타난 결과를 파악하는 데 도움을 줍니다.

실제 실습에서 나온 정확한 메시지와 시각은 직접 확인한 로그를 기준으로 작성해야 합니다.

9. 반드시 기억할 내용

- 로그는 요청 로그, 컨테이너 로그, 시스템 로그 세 종류로 구분합니다.
- 빌드 문제는 Cloud Build 기록도 확인합니다.
- 응답이 느린 문제는 우선 요청 로그의 지연 시간부터 확인합니다.
- 앱 시작 실패는 컨테이너 로그의 오류 메시지와 시스템 로그를 함께 봅니다.

- 배포 직후에는 리비전 이름을 확인해 어느 버전의 로그인지 구분합니다.
- 코딩 에이전트에게는 증상, 시간, 서비스·리비전, 근거 로그를 전달합니다.
- 로그를 읽고 원인을 확인하기 전에는 코드를 수정하지 않습니다.
- 로그에는 민감한 정보나 사용자 원문을 불필요하게 남기지 않습니다.

10. 한 문장 정리

로그는 단순한 오류 기록이 아니라, AI 서비스에서 무슨 일이 일어났는지 증거를 모아 재구성하고 코딩 에이전트가 정확하게 문제를 해결하도록 돕는 운영 도구입니다.

Chapter07

/강의정리 챕터 07. 서로 써 보기

1. 오늘 강의의 목적

오늘 강의의 핵심은 내가 만든 AI 서비스를 다른 사람이 실제로 사용하게 했을 때, 비용·성능·개인정보·운영 측면에서 어떤 문제가 발생하는지 직접 확인하는 것입니다.

지금까지는 Cloud Run에 서비스를 배포하고, 타임아웃과 인스턴스 수를 설정하고, 로그에서 오류 원인을 찾는 방법을 배웠습니다. 이번에는 이 지식을 실제 사용자 관점에서 종합해 봅니다.

특히 다음 네 가지를 익힙니다.

1. 다른 사람이 내 서비스에 접속했을 때 요청과 비용이 어떻게 발생하는지 확인합니다.
2. 동시성과 최대 인스턴스 설정이 응답 속도와 처리량에 어떤 영향을 주는지 측정합니다.
3. 새로운 버전을 배포하고 이전 버전으로 되돌리는 방법을 익힙니다.
4. 사용자가 자신의 LLM API 키를 사용하는 BYOK 구조를 이해합니다.

2. 중학생도 이해할 수 있는 핵심 개념

2-1. 서비스 URL을 공개하면 누구나 사용할 수 있습니다

내가 만든 AI 서비스를 식당이라고 생각해 보겠습니다.

- 서비스 URL은 식당 주소입니다.
- 사용자는 주소를 알고 있으면 방문할 수 있습니다.

- LLM API 키는 식재료를 구입하는 데 사용하는 내 결제 수단과 비슷합니다.
- 다른 사람이 음식을 주문해도 식재료 비용은 식당 주인이 부담할 수 있습니다.

따라서 서비스 URL을 공개했다고 해서 다른 사람의 사용 비용까지 다른 사람이 부담하는 것은 아닙니다.

다른 사용자가 내 서비스에 접속

Cloud Run이 요청을 처리

내 서버가 내 LLM API 키로 모델 호출

Cloud Run 및 LLM API 비용 발생 가능

그래서 공개 서비스에서는 누가 얼마나 사용할 수 있는지, 비용이 어디까지 발생할 수 있는지를 함께 관리해야 합니다.

2-2. 최대 인스턴스와 동시성은 서로 다른 설정입니다

두 설정을 건물에 비유하면 이해하기 쉽습니다.

- 최대 인스턴스(max instances): 건물의 최대 개수입니다.
- 동시성(concurrency): 건물 하나에서 동시에 처리할 수 있는 요청 수를 정하는 설정입니다.

예를 들어 동시성이 80이고 최대 인스턴스가 3이라면, 설정상 인스턴스 3개가 각각 최대 80개의 요청을 동시에 처리할 수 있습니다. 단순 계산으로는 240개의 동시 요청에 해당하지만, 실제 처리량과 지연 시간은 앱의 구조, CPU 사용량, 외부 API 응답 시간, 플랫폼 동작에 따라 달라집니다.

설정	의미	예상되는 특징
동시성 80, 최대 3	인스턴스 하나가 여러 요청 처리	인스턴스 수가 적게 늘어날 수 있음
동시성 1, 최대 20	요청 하나당 인스턴스 하나 수준으로 처리	요청이 몰리면 인스턴스가 더 많이 필요할 수 있음
동시성 1, 최대 3	인스턴스 수를 낮게 제한	요청이 몰리면 대기하거나 일부 요청이 거절될 수 있음

중요한 점은 동시성을 낮추면 반드시 빨라지는 것도, 높이면 반드시 비용이 줄어드는 것도 아니라는 사실입니다. 직접 측정해서 판단해야 합니다.

2-3. 콜드 스타트란 무엇인가요?

Cloud Run은 요청이 없을 때 인스턴스를 줄이거나 종료했다가, 새로운 요청이 들어오면 인스턴스를 다시 시작할 수 있습니다. 이때 새 인스턴스를 준비하는 데 걸리는 시간이 콜드 스

타트(cold start) 입니다.

비유하면 손님이 없어서 주방을 닫아 둔 식당에 갑자기 손님이 왔을 때, 주방을 다시 열고 조리 준비를 하는 과정과 같습니다.

첫 번째 응답만 유독 늦었다면 콜드 스타트를 의심할 수 있습니다. 다만 실제 원인을 확인하려면 다음 요소를 함께 살펴봐야 합니다.

- 요청 로그의 지연 시간
- 컨테이너 인스턴스 수와 시작 관련 시스템 로그
- LLM API 응답 시간
- 애플리케이션 내부의 처리 시간

요청이 느렸다는 사실만으로 콜드 스타트라고 단정해서는 안 됩니다.

3. 비유로 이해하기: 내 서비스는 작은 온라인 식당입니다



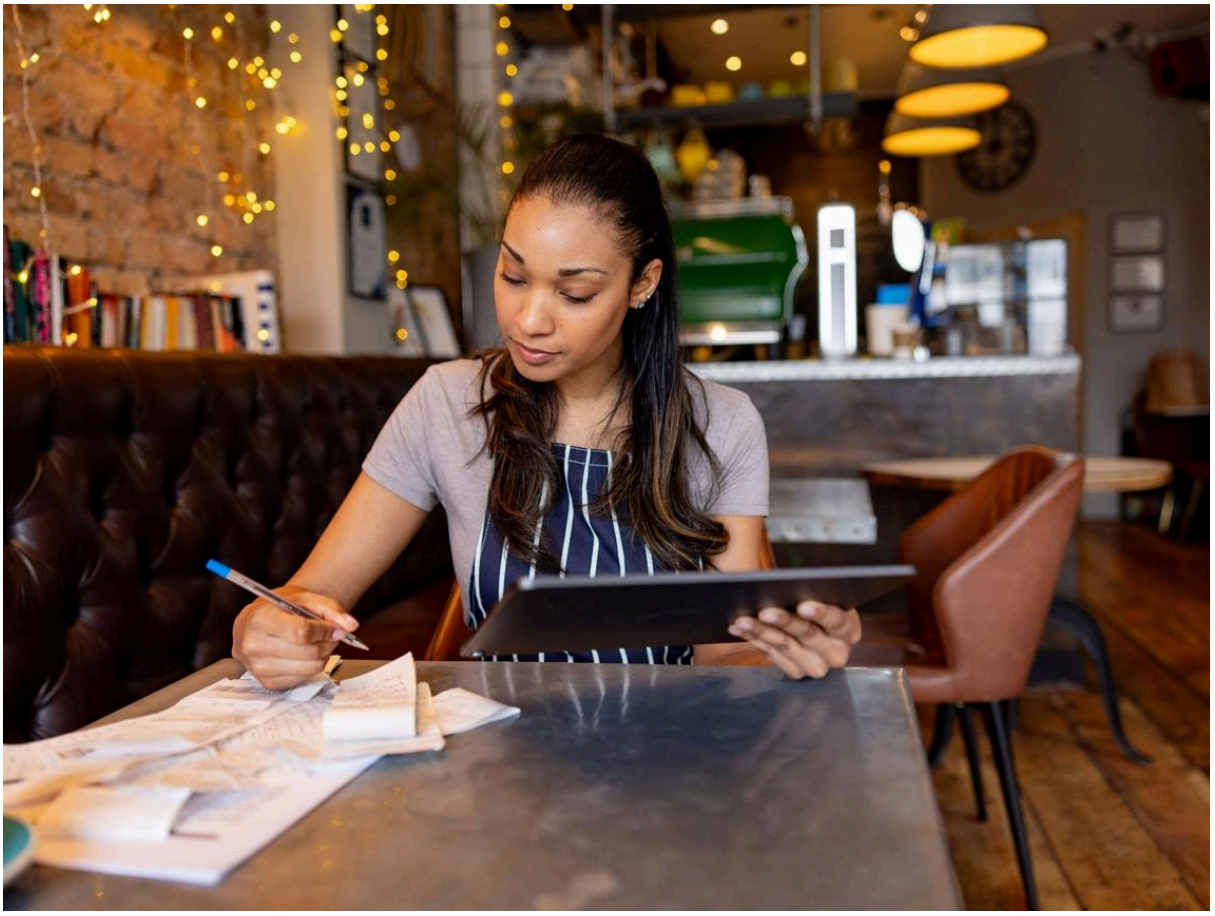
① 서비스 공개와 다른 사람의 접속

식당 주소를 알려 주면 다른 사람이 방문할 수 있습니다. 내 서비스도 URL이 공개되어 있으면 다른 사람이 요청을 보낼 수 있습니다.



② 동시성과 최대 인스턴스

주방 하나가 손님 여러 명의 주문을 처리할 수도 있고, 주방을 여러 개 열 수도 있습니다. 주방을 무작정 늘리면 운영비가 증가하고, 너무 적게 열면 대기 시간이 길어질 수 있습니다.



③ 로그와 측정항목

주문 기록과 운영 기록을 확인하면 어느 시간에 손님이 몰렸는지, 주문이 왜 늦어졌는지 알 수 있습니다.

운영자는 손님이 많아졌다는 사실만 보는 것이 아니라, 손님 수·처리 속도·비용을 함께 보고 적절한 운영 방식을 선택해야 합니다.

4. 실습별 핵심 정리

4-1. 실습 ① 다른 사람의 요청이 내 서비스에 남는지 확인하기

다른 사람에게 내 서비스 URL을 알려 주고, 상대방이 접속해 캐릭터와 대화하도록 합니다.

그다음 Cloud Run의 로그와 측정항목을 확인합니다.

- Logs 탭에서 `/api/chat` 요청을 검색합니다.
- 상대방이 접속한 시각의 요청 로그를 찾아 지연 시간을 확인합니다.
- Metrics 탭에서 같은 시간대에 요청 수가 증가했는지 확인합니다.

이 실습에서 알아야 할 사실은 내가 직접 사용하지 않은 요청도 내 서비스의 로그와 사용량에 기록될 수 있다는 것입니다.

또한 현재 예제 앱은 대화 내용을 인스턴스 메모리에 저장합니다. 로그인과 사용자별 데이터 분리가 없는 구조에서는 여러 사용자의 캐릭터나 대화 상태가 섞일 수 있습니다.

실제 서비스로 발전시키려면 다음이 필요합니다.

- 사용자 식별과 인증
- 사용자별 데이터 접근 권한
- 데이터베이스를 이용한 대화 기록 저장
- 요청 횟수 및 비용 제한
- 개인정보와 대화 원문의 안전한 처리

4-2. 실습 ② 배포와 롤백

코드나 화면 문구를 수정해 새 리비전을 배포한 다음, 이전 리비전으로 트래픽을 돌렸다가 최신 리비전으로 복귀하는 실습입니다.

```
# 현재 리비전 목록 확인
gcloud run revisions list \
  --service friend-agent \
  --region us-central1

# 소스 배포
gcloud run deploy friend-agent \
  --source . \
  --region us-central1

# 이전 리비전으로 트래픽 전환
gcloud run services update-traffic friend-agent \
  --region us-central1 \
  --to-revisions <이전_리비전_이름>=100

# 최신 리비전으로 복귀
gcloud run services update-traffic friend-agent \
  --region us-central1 \
  --to-latest
```

여기서 `update-traffic`은 코드를 다시 빌드하는 명령이 아니라 어느 리비전으로 사용자 요청을 보낼지 조정하는 명령입니다.

실습할 때는 다음 세 가지를 확인합니다.

1. 새 리비전이 정상적으로 생성되었는가?

2. 트래픽이 의도한 리비전으로 100% 전달되는가?
3. 요청 로그에 실제로 어느 리비전이 기록되는가?

운영 서비스에서는 이전 리비전이 남아 있다는 사실만으로 롤백이 완전히 안전하다고 볼 수 없습니다. 이전 리비전이 정상 상태인지, 데이터베이스 변경과 설정 변경이 호환되는지도 확인해야 합니다.

4-3. 실습 ③ 부하 테스트로 실제 성능 비교하기

이번에는 LLM API를 호출하지 않는 별도의 `load-test` 서비스를 만들어 동시 요청 50개를 보냅니다. 각 요청이 약 2초 동안 기다리도록 만들어 인스턴스 확장과 대기 현상을 관찰합니다.

설정	동시성	최대 인스턴스	실험 목적
A	80	3	한 인스턴스가 많은 요청을 처리할 수 있는지 확인
B	1	20	요청이 몰릴 때 인스턴스가 얼마나 확장되는지 확인
C	1	3	인스턴스 상한이 처리 지연이나 오류에 미치는 영향 확인

각 설정에서 다음 값을 기록합니다.

- HTTP 200 응답 수
- 200이 아닌 응답 수
- p50 응답 시간
- p95 응답 시간
- 최대 응답 시간
- 실제 관측된 최대 인스턴스 수

p50은 요청의 절반 정도가 이 시간 안에 완료되었다는 뜻이고, p95는 약 95%의 요청이 이 시간 안에 완료되었다는 뜻입니다. 부하 테스트 결과를 비교할 때 평균만 보지 않고 p95와 최대 응답 시간도 살펴야 느린 요청을 파악하기 쉽습니다.

예상 결과를 실제 측정 결과와 혼동하지 마세요. 설정 B에서 인스턴스가 많이 늘어날 수 있지만, 실제 확장 속도와 응답 시간은 환경에 따라 달라집니다. 또한 200이 아닌 응답이 나타났다면 요청 로그에서 정확한 상태 코드와 원인을 확인해야 합니다.

실습이 끝나면 테스트용 서비스는 삭제합니다.

```
gcloud run services delete load-test \
  --region us-central1
```

4-4. 실습 ④ BYOK 구조로 변경하기

BYOK는 Bring Your Own Key의 약자로, 사용자가 자신의 LLM API 키를 입력해 서비스를 이용하는 방식입니다.

기존 구조에서는 운영자가 가진 키로 모든 사용자의 요청을 처리할 수 있습니다. BYOK에서는 각 사용자가 자신의 키를 사용하므로 운영자의 모델 API 비용 부담을 줄일 수 있습니다.

항목	운영자 키 사용	BYOK
LLM API 비용	주로 운영자 부담	사용자의 키와 계정에 청구
사용자의 준비	별도 키 불필요	키 발급과 입력 필요
운영자 부담	API 비용 및 사용량 관리	키 보안 및 사용자 안내 필요
사용 편의성	상대적으로 높음	상대적으로 낮을 수 있음

강의에서 제안한 구조는 브라우저의 `sessionStorage`에 키를 보관하고, `/api/chat` 요청마다 `X-User-LLM-Key` 헤더로 전달하는 방식입니다.

다만 sessionStorage에 저장한다고 해서 키가 완전히 안전해지는 것은 아닙니다. 악성 스크립트가 실행되는 상황에서는 키가 노출될 수 있으므로 입력값 검증, XSS 방어, HTTPS 사용, 민감한 로그 차단이 중요합니다. 서버에서도 키를 요청 처리 중에만 사용하고, 예외 메시지나 오류 추적 정보에 포함하지 않도록 주의해야 합니다.

또한 키를 지웠을 때 운영자 키로 조용히 전환하는 방식은 사용자의 의도와 비용 부담을 혼동시킬 수 있습니다. 키가 없으면 명확히 안내하고, 잘못된 키나 한도 초과는 이해할 수 있는 메시지로 알려 주는 것이 좋습니다.

5. 오늘 배운 내용을 내 프로젝트에 적용한다면

사용자님의 실제 프로젝트 중에서는 MSM과 뉴스레터 에이전트에 다음과 같이 적용할 수 있습니다.

MSM — My-Story-Maker

사용자가 직접 글을 넣고 AI로 이야기를 만드는 서비스

1. 사용자 분리: A 사용자의 원문과 B 사용자의 원문이 섞이지 않도록 로그인과 사용자별 데이터 접근 권한을 설계합니다.

2. 성능 측정: 글 분석, 원문 보정, 웹소설 변환 단계별로 처리 시간을 기록합니다. 어느 단계가 느린지 파악할 수 있습니다.
3. 비용 관리: 한 사용자가 긴 글을 반복해서 변환할 때 비용이 급증하지 않도록 글자 수, 요청 횟수, 모델 호출 횟수에 제한을 둡니다.
4. 안전한 배포: 새 버전이 문체 분석이나 원문 보존 기능을 망가뜨리면 이전 리비전으로 트래픽을 되돌릴 수 있도록 준비합니다.

뉴스레터 자동화 에이전트

뉴스 수집부터 검증과 발행까지 연결하는 자동화

1. 단계별 기록: `collect → select → research → verify → publish` 각 단계의 성공 여부와 처리 건수를 기록합니다.
2. 실패 지점 파악: 뉴스 수집은 성공했지만 검증에서 실패했다면 전체 파이프라인을 다시 실행하기 전에 해당 단계의 원인을 확인합니다.
3. 운영 비용 통제: 요청량뿐 아니라 뉴스 한 건당 LLM 호출 수와 토큰 사용량을 확인합니다.
4. 배포 검증: 새 버전에서 결과 형식이나 선별 기준이 달라지지 않았는지 확인한 후 정식으로 전환합니다.

사용자님이 만들고자 하는 1인 AI 사업에서는 기능을 만드는 능력만큼이나 사용량을 통제하고, 문제를 진단하고, 안전하게 서비스를 유지하는 능력이 중요합니다.

6. 오늘 강의에서 반드시 기억할 내용

- 공개 URL은 다른 사람의 요청을 받을 수 있는 통로입니다.
- 운영자 키를 사용하는 서비스에서는 다른 사람의 요청으로도 운영자의 LLM API 비용이 발생할 수 있습니다.
- 최대 인스턴스는 확장 규모를 제한하지만 LLM API 비용을 직접 제한하지는 않습니다.
- 동시성과 최대 인스턴스는 실제 부하 테스트 결과를 보고 조정해야 합니다.
- p50뿐 아니라 p95와 최대 응답 시간도 확인해야 합니다.
- 리비전 전환과 롤백은 운영 중 장애에 대응하는 중요한 방법입니다.
- 여러 사용자가 이용하는 서비스는 사용자별 데이터 분리와 인증을 고려해야 합니다.
- BYOK를 적용하더라도 API 키의 보안과 사용자의 비용 안내는 여전히 중요합니다.

7. 한 문장 정리

AI 서비스를 배포하는 것에서 한 걸음 더 나아가, 다른 사람이 사용해도 비용·성능·데이터를 통제하고 문제가 생기면 안전하게 복구할 수 있는 운영 능력을 갖추는 것이 오늘 강의의 핵심입니다.

Chapter08

/강의정리 챕터 08. 오늘 배운 것 — 회고 퀴즈

1. 오늘 강의의 핵심

이번 회고 문제는 공개된 AI 서비스를 운영할 때 걱정되는 문제 세 가지를 떠올리고, 지금까지 배운 설정으로 어디까지 예방할 수 있고 무엇은 추가로 설계해야 하는지 구분하는 문제입니다.

대표적인 걱정은 다음 세 가지입니다.

1. 비용 폭증: 다른 사람이 내 서비스를 반복해서 사용해 LLM API 비용이 커지는 문제
2. 서비스 지연 및 장애: 요청이 몰리거나 응답이 늦어져 서비스가 느려지거나 실패하는 문제
3. 대화 데이터 유실 또는 혼합: 인스턴스가 바뀌면서 대화가 사라지거나 여러 사용자의 데이터가 섞이는 문제

2. 주관식 퀴즈 모범 답안

걱정되는 문제	오늘 배운 설정 중 도움이 되는 것	아직 막지 못하는 것
① 다른 사람이 서비스를 많이 사용해 비용이 폭증함	최대 인스턴스 설정으로 Cloud Run 인스턴스 규모를 제한하고, LLM 제공사의 사용 한도 설정으로 API 사용을 제한할 수 있습니다.	최대 인스턴스만으로 LLM API 비용을 제한할 수는 없습니다. 요청별 호출 수와 토큰 사용량도 관리해야 하며, 인증·사용자별 이용 제한도 별도로 설계해야 합니다.
② 요청이 몰리거나 응답이 늦어져 서비스가 느려짐	타임아웃, 최대 인스턴스, 동시성 설정으로 처리 방식과 확장 규모를 조절합니다. 로그와 측정항목으로 지연 시간과 오류를 확인합니다.	타임아웃은 느린 요청을 무조건 빠르게 만들지 않습니다. 최대 인스턴스를 제한하면 요청이 대기하거나 거절될 수도 있습니다. 실제 부하 테스트와 성능 개선이 필요합니다.
③ 대화가 사라지거나 다른 사용자의 대화와	로그로 오류와 리비전 변경 시점을 확인하고, 인스턴스 교체나 배포가	로그는 데이터를 복구하거나 사용자별 데이터를 분리해 주지 않습니다. 대화

걱정되는 문제	오늘 배운 설정 중 도움이 되는 것	아직 막지 못하는 것
섞임	있었는지 추적할 수 있습니다.	기록을 데이터베이스에 저장하고, 로그인·사용자 식별·접근 권한을 구현해야 합니다.

3. 반드시 구분해야 할 세 가지

비용 통제

최대 인스턴스는 Cloud Run의 규모를 제한하는 설정이고, LLM 제공사의 사용 한도는 모델 API 비용을 통제하는 데 도움이 됩니다. Google Cloud 예산 알림은 지출을 알려 주는 기능이지, 그 자체로 지출을 중단시키는 기능은 아닙니다.

성능과 안정성

타임아웃은 요청이 너무 오래 지속되는 것을 제한하고, 동시성과 인스턴스 설정은 요청 처리 방식을 조절합니다. 설정만으로 빠른 응답이나 무중단 운영이 보장되지는 않습니다.

데이터 보호

로그는 문제의 원인을 추적하는 도구입니다. 데이터 보존과 사용자별 분리는 별도로 설계해야 하며, 중요한 대화 기록은 인스턴스 메모리에만 저장하지 않는 것이 좋습니다.

4. AI 서비스 운영자로서의 교훈

이번 강의에서 가장 중요한 것은 설정 하나가 모든 문제를 해결해 주지는 않는다는 사실입니다.

- 비용 문제는 인스턴스 제한과 LLM 사용 한도를 함께 관리해야 합니다.
- 성능 문제는 타임아웃만 조정하지 말고 실제 지연 시간과 부하 테스트 결과를 확인해야 합니다.
- 데이터 문제는 로그를 잘 읽는 것만으로 해결되지 않으며, 데이터베이스와 사용자별 권한 구조가 필요합니다.

사용자님이 개발 중인 MSM이나 뉴스레터 에이전트도 마찬가지입니다. 기능이 정상적으로 동작하는 것뿐 아니라, 사용량이 증가해도 비용을 통제할 수 있는지, 장애 원인을 찾을 수 있는지, 사용자 데이터가 안전하게 보존되는지까지 설계해야 실제 서비스로 발전시킬 수 있습니다.

5. 한 문장 정리

AI 서비스를 안전하게 운영하려면 비용·성능·데이터 문제를 구분하고, 각 문제를 막아 주는 설정과 별도로 구현해야 할 기능을 정확히 알아야 합니다.